TIỂU LUẬN MÔN CƠ SỞ DỮ LIỆU

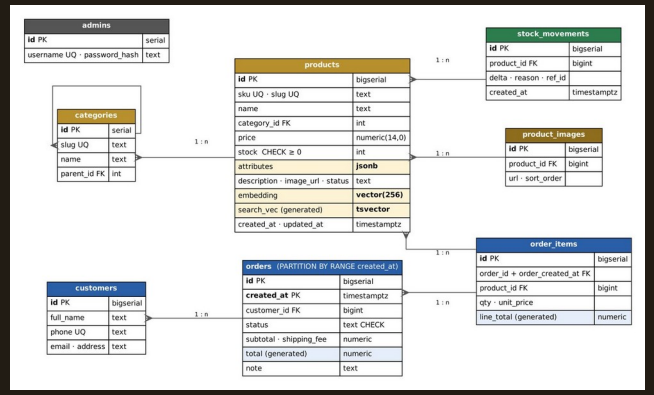
Xây dựng hệ thống bán hàng trang sức với PostgreSQL

Nhóm 6

Phùng Anh Tuấn – 2541864024

Võ Bùi Duy Thông – 2541864019

Trường Đại học Công nghệ TP.HCM · Viện Đào tạo Sau đại học · 10/2026



Vì sao chọn PostgreSQL?

#4

bảng xếp hạng DB-Engines
(sau Oracle, MySQL, SQL Server)

+21,97

điểm nửa đầu 2026 – tăng mạnh nhất
trong khi top 3 đều giảm

4 lần

DBMS of the Year
(2017, 2018, 2020, 2023)

100%

mã nguồn mở, giấy phép tự do
không có bản "Enterprise"

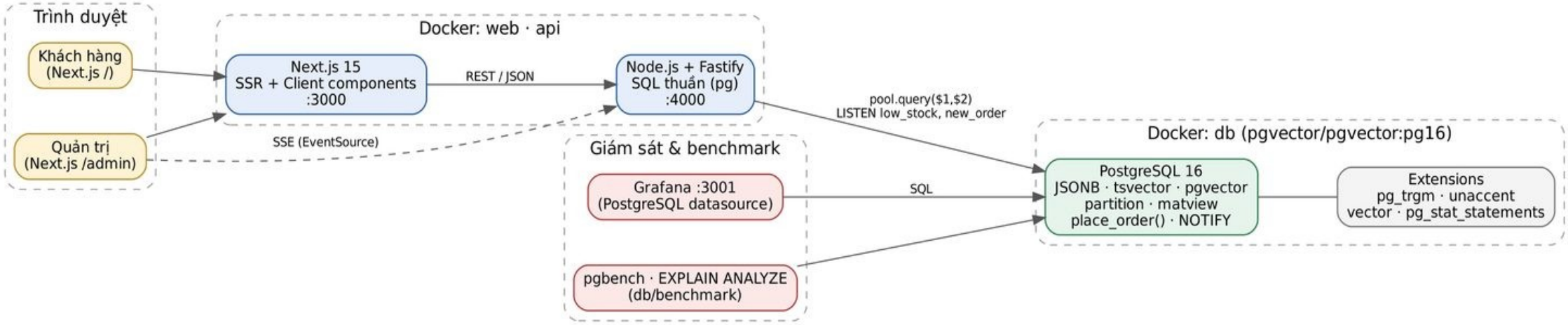
CSDL quan hệ + đa mô hình trong một hệ thống

- JSONB, mảng, full-text search, pgvector… ngay trong SQL chuẩn
- 6 loại index: B-tree, Hash, GIN, GiST, SP-GiST, BRIN
- Partition khai báo, materialized view, trigger, PL/pgSQL, LISTEN/NOTIFY
- Cách làm của nhóm: mỗi tính năng gắn 1 nghiệp vụ, có số đo

6 đặc thù nghiệp vụ → 6 tính năng PostgreSQL

Đặc thù nghiệp vụ	Tính năng PostgreSQL
1. Mỗi loại trang sức có thuộc tính khác nhau (ni, carat, dài dây…)	**JSONB + GIN index**
2. Khách gõ không dấu, sai chính tả vẫn phải tìm ra	**Full-text search + unaccent + pg_trgm**
3. Gợi ý “sản phẩm giống” khi xem chi tiết	**pgvector + HNSW**
4. Đơn hàng tăng mãi theo thời gian	**Partitioning theo tháng + BRIN**
5. Tồn kho ít, 2 khách mua cùng lúc chiếc cuối	**Transaction + SELECT … FOR UPDATE**
6. Dashboard nhanh, báo hết hàng tức thì	**Materialized view + LISTEN/NOTIFY**

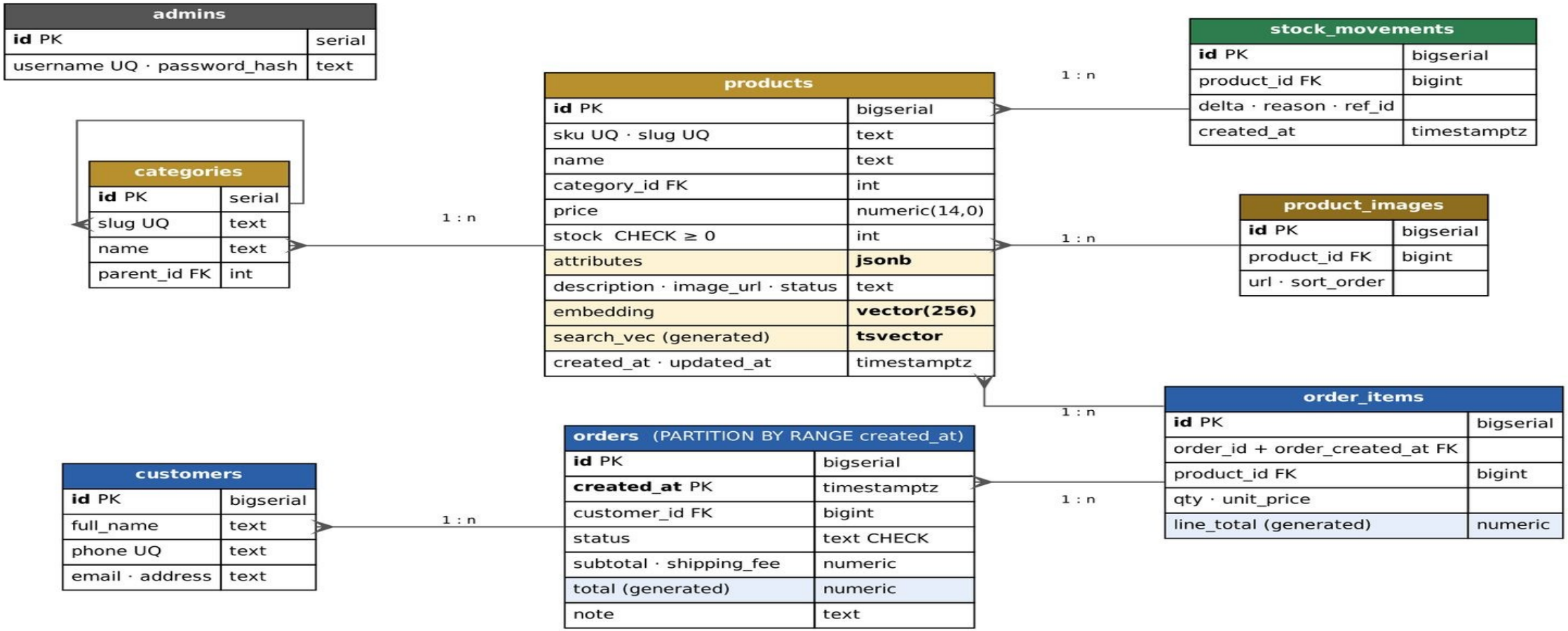
Kiến trúc hệ thống



PostgreSQL 16 + pgvector | **Node.js + Fastify · SQL thuần, không ORM** | **Next.js 15 (khách + admin)**

- Logic quan trọng nằm trong CSDL: hàm place_order(), cancel_order(), trigger NOTIFY
- 6 file migration SQL, trình chạy migration tự viết 60 dòng, mỗi file = 1 transaction
- docker compose up → db + api + web + Grafana; dữ liệu mẫu 3.033 sản phẩm, 30.000 đơn

Thiết kế CSDL: 8 bảng



Cột tô màu = nơi áp dụng tính năng đặc biệt của PostgreSQL

1 JSONB – thuộc tính sản phẩm linh hoạt

```
attributes JSONB NOT NULL DEFAULT '{}'
CREATE INDEX idx_products_attributes
  ON products USING GIN (attributes jsonb_path_ops);

-- Nhẫn
{"metal":"Vàng trắng 18K","ring_size":12,
 "stone":"Kim cương","carat":0.5,"clarity":"VS1"}
-- Dây chuyền
{"metal":"Vàng 24K","length_cm":50,"style":"Xích"}

-- Lọc: chất liệu + đá, 1 câu SQL cho mọi tổ hợp
WHERE attributes @> '{"metal":"Vàng
18K","stone":"Kim cương"}'
```

28×

nhanh hơn quét tuần tự
0,056 ms so với 1,57 ms

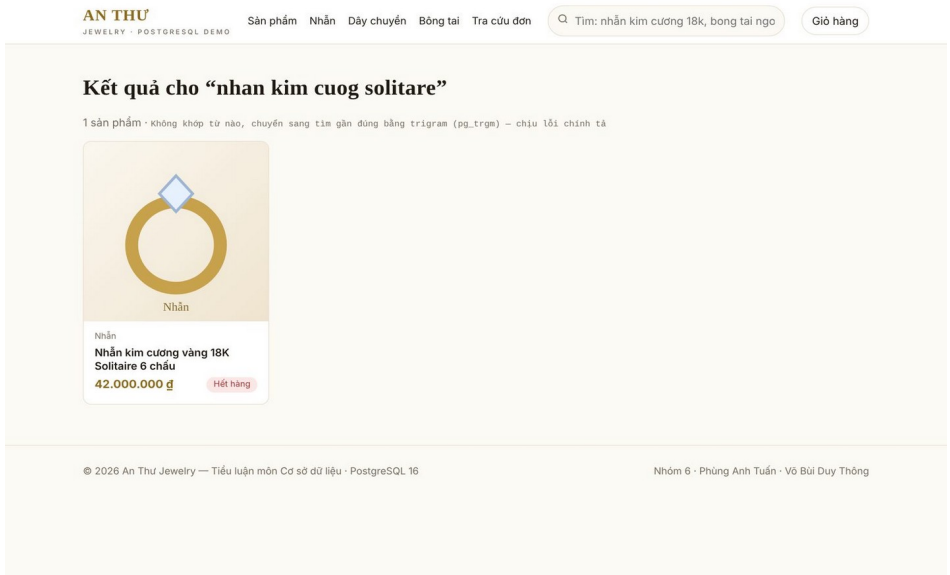
- Không ALTER TABLE khi thêm loại trang sức mới
- Admin thêm/xoá thuộc tính ngay trên giao diện
- Bộ lọc bên trái cũng sinh từ JSONB

2 Full-text search tiếng Việt: không dấu, sai chính tả vẫn tìm ra

```
-- Hàm bỏ dấu IMMUTABLE để dùng trong index
CREATE FUNCTION vi_unaccent(text) ... IMMUTABLE
  AS $$ SELECT lower(unaccent($1)) $$;

-- tsvector sinh tự động (generated column)
search_vec TSVECTOR GENERATED ALWAYS AS (
  to_tsvector('simple', vi_unaccent(name)) ...) STORED;
CREATE INDEX ... USING GIN (search_vec);
CREATE INDEX ... USING GIN (vi_unaccent(name)
gin_trgm_ops);

-- Tầng 1: search_vec @@
plainto_tsquery(vi_unaccent($1))
-- Tầng 2: vi_unaccent(name) % 'nhan kim cuog solitare'
```



“nhan kim cuog solitare” → Nhẫn kim cương vàng 18K Solitaire (trigram, similarity 0,44)

FTS nhanh hơn 7×

Gợi ý khi gõ: nha:* & kim:*

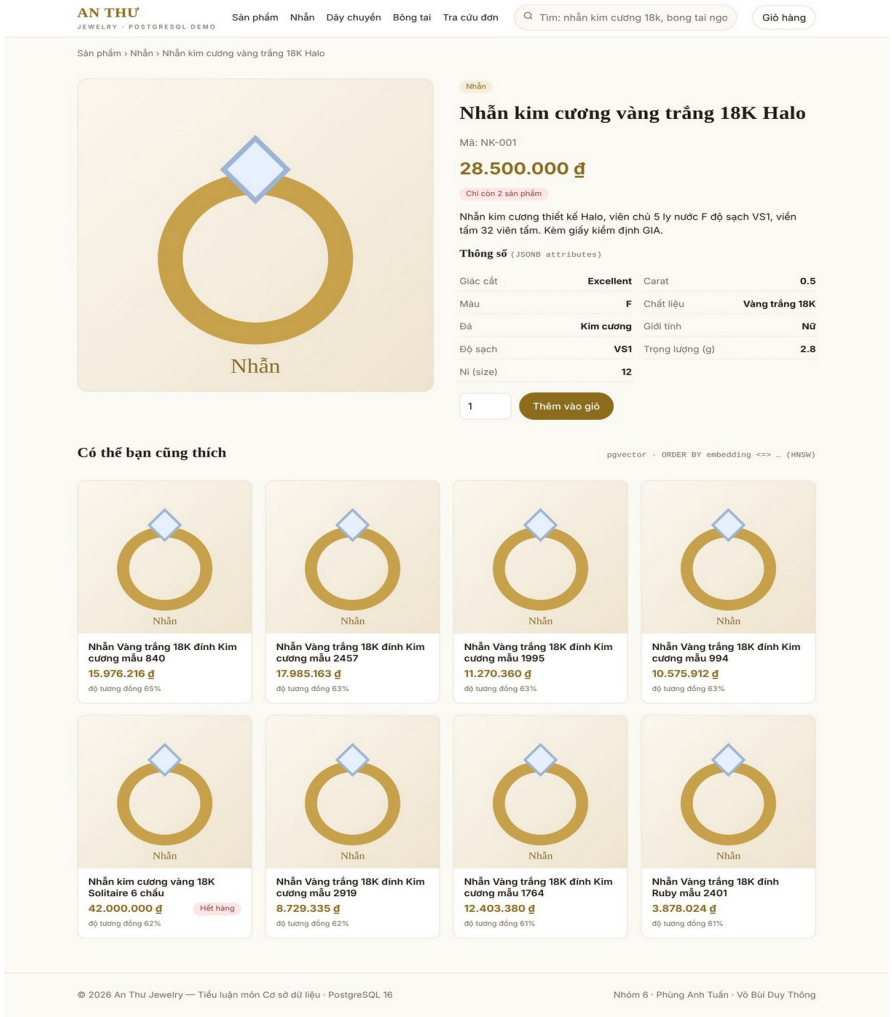
Không cần Elasticsearch

3 pgvector – gợi ý sản phẩm tương tự (AI trong CSDL)

```
embedding vector(256)
CREATE INDEX idx_products_embedding
  ON products USING hnsw (embedding vector_cosine_ops);

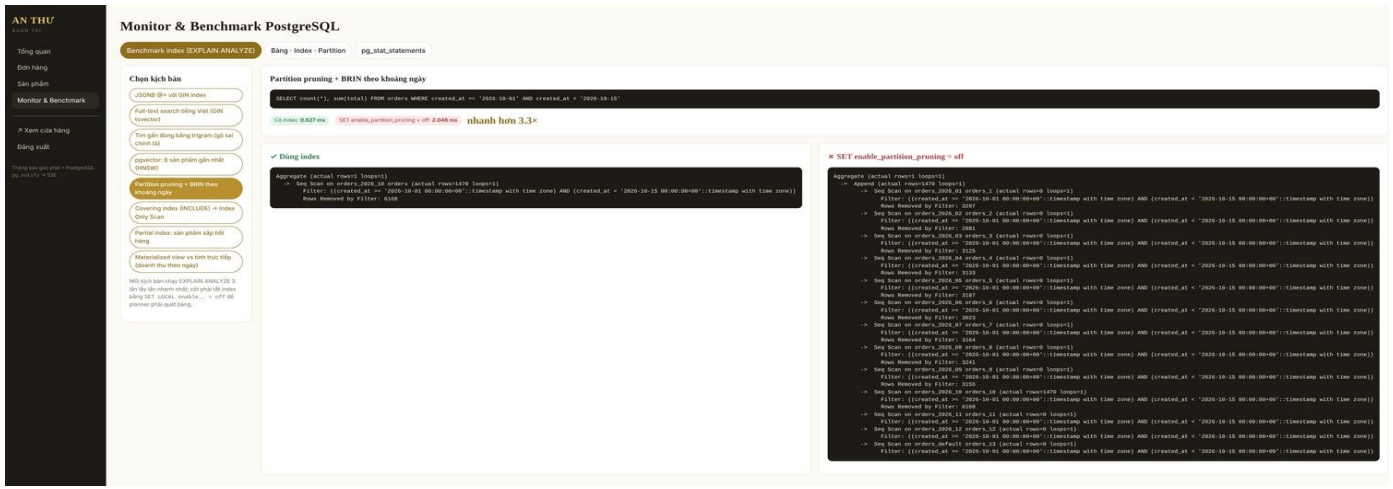
-- 8 sản phẩm gần nhất, <=> = khoảng cách cosine
SELECT name, 1 - (p.embedding <=> me.embedding) AS
similarity
FROM products p, (SELECT embedding FROM products WHERE
slug=$1) me
ORDER BY p.embedding <=> me.embedding
LIMIT 8;
```

- Embedding demo: feature hashing từ tên + danh mục + thuộc tính JSONB (256 chiều, không cần API ngoài)
- Thay bằng OpenAI / CLIP sau này: chỉ đổi 1 hàm, schema và SQL giữ nguyên
- HNSW: index Scan 0,25 ms so với 1,43 ms tính toàn bộ 3.033 vector



4 Partitioning theo tháng + BRIN cho bảng đơn hàng

```
CREATE TABLE orders (
  id BIGSERIAL, created_at TIMESTAMPTZ NOT NULL, ...
  PRIMARY KEY (id, created_at)
) PARTITION BY RANGE (created_at);
CREATE TABLE orders_2026_10 PARTITION OF orders
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
CREATE INDEX ... ON orders USING BRIN (created_at);
```



1 / 13

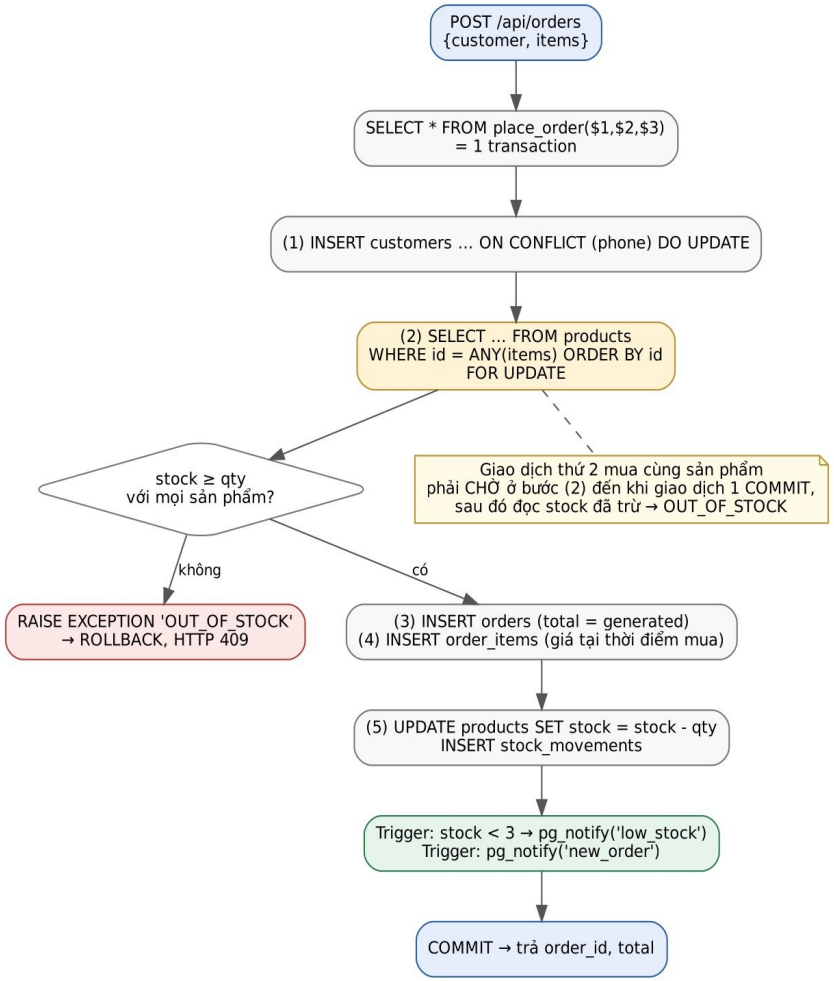
partition được quét
nhờ partition pruning

312 kB

BRIN so với 1.000 kB B-tree
cùng cột created_at

- Lưu trữ đơn cũ = DETACH 1 partition
- Tạo tháng mới: ensure_orders_partition()

5 Transaction + SELECT … FOR UPDATE – không bán quá tồn kho




```
-- place_order(): 1 hàm PL/pgSQL = 1 transaction
FOR r IN
  SELECT p.id, p.name, p.stock, i.qty
  FROM jsonb_to_recordset(p_items) i JOIN products p ...
  ORDER BY p.id     -- khoá cùng thứ tự → không deadlock
  FOR UPDATE OF p   -- giao dịch 2 phải CHỜ giao dịch 1
LOOP
  IF r.stock < r.qty THEN
    RAISE EXCEPTION 'OUT_OF_STOCK: "%" chỉ còn %', ...;
  END IF;
END LOOP;
-- INSERT orders, order_items; UPDATE stock;
CHECK(stock>=0)
```

5 / 50

client mua được khi kho = 5
45 client bị OUT_OF_STOCK

625

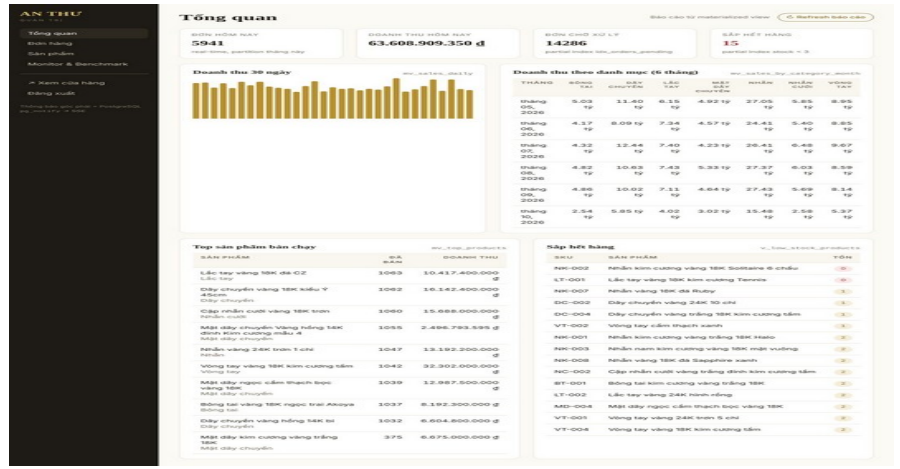
đơn / giây với 30 client
(pgbench, máy ảo 2 vCPU)

6 Materialized view + LISTEN/NOTIFY cho dashboard

```
CREATE MATERIALIZED VIEW mv_sales_daily AS
SELECT date_trunc('day', created_at)::date AS day,
       count(*) AS orders, sum(total) AS revenue ...
CREATE UNIQUE INDEX ON mv_sales_daily (day);
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_sales_daily;

-- Trigger: tồn < 3 → pg_notify('low_stock', json)
-- Node.js: LISTEN low_stock → Server-Sent Events → trình
duyệt admin
```





150×

doanh thu 30 ngày: 0,012 ms
so với 1,8 ms tính trực tiếp

< 0,1 s

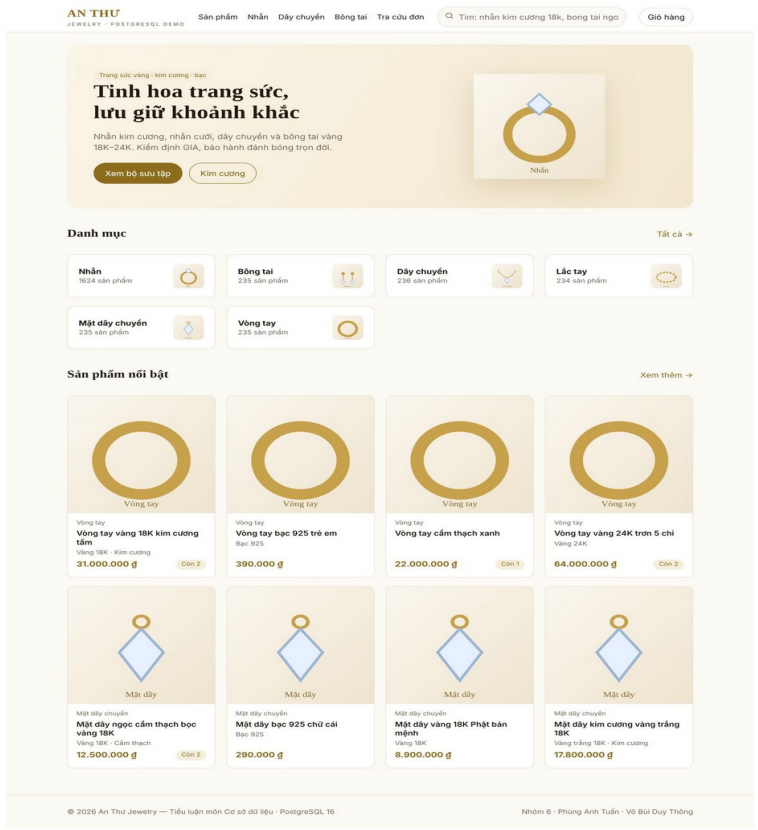
cảnh báo tới admin
không F5, không Redis

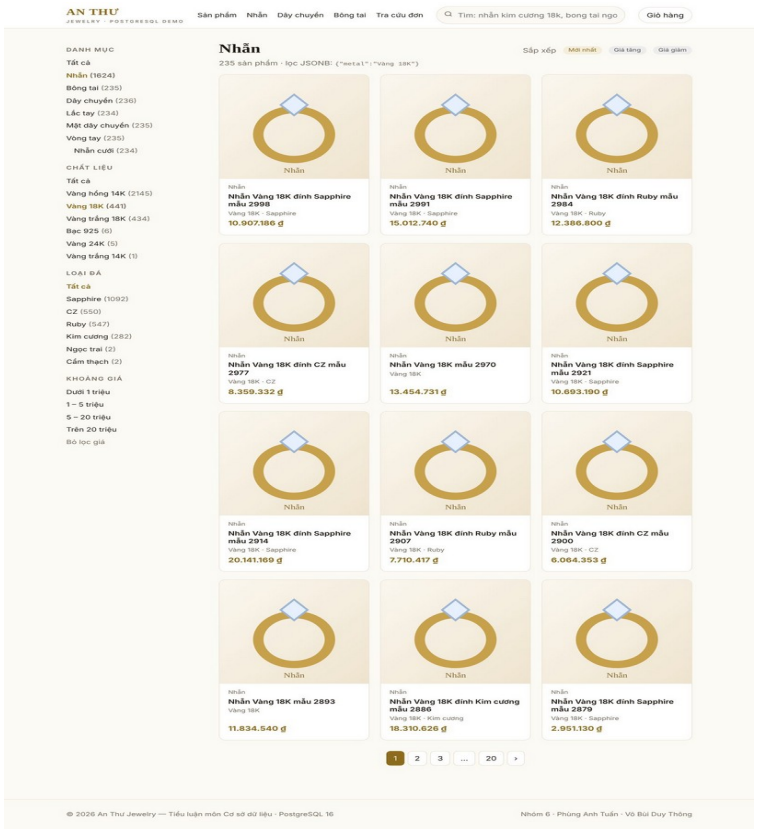
Các index và cột đặc biệt khác

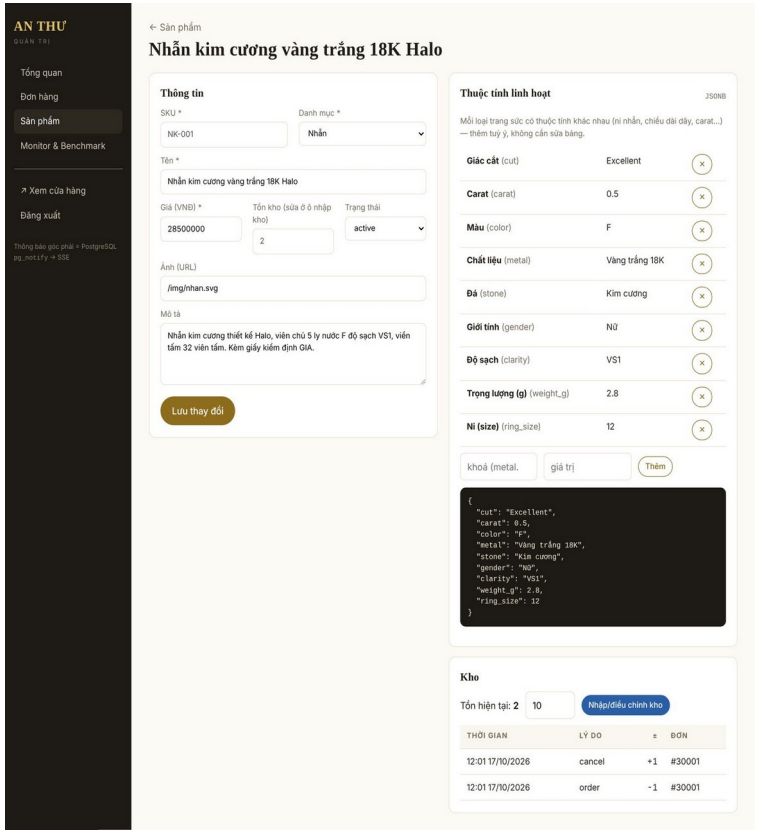
Loại	Trong hệ thống	Lợi ích đo được
Partial index	`products (stock, id) WHERE stock < 3`	40 kB thay vì 168 kB; 0,018 ms (16×)
Covering index (INCLUDE)	`orders (customer_id, created_at) INCLUDE (status, total)`	Index Only Scan, Heap Fetches = 0
Expression index	`customers (lower(email))`	Tìm email không phân biệt hoa thường
Generated column	`total = subtotal + shipping_fee` `search_vec = to_tsvector(...)`	CSDL tự tính, app không ghi sai được
GIN / HNSW / BRIN	`JSONB, tsvector, trigram, vector, thời gian`	2,5× – 28× so với quét tuần tự

Mỗi index gắn với đúng 1 truy vấn của ứng dụng – theo dõi lượt dùng bằng pg_stat_user_indexes

Ứng dụng: trang bán hàng và trang quản trị

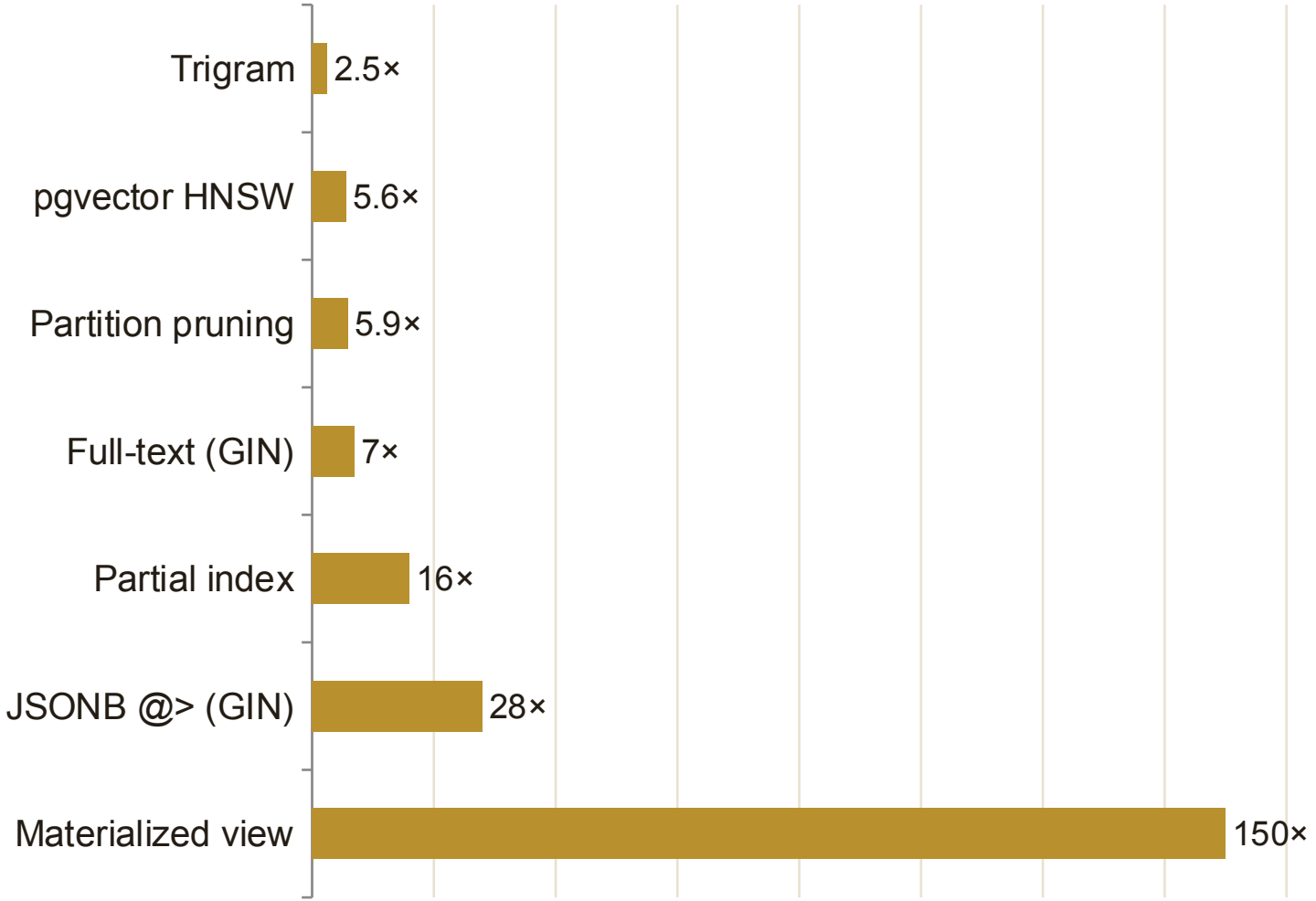






Trang chủ · Lọc JSONB · Admin sửa thuộc tính & nhập kho — demo trực tiếp: tìm kiếm, đặt hàng, cảnh báo real-time, benchmark EXPLAIN

Benchmark: có index nhanh hơn bao nhiêu lần?



1.690

truy vấn đọc / giây
(FTS + JSONB + vector)

625

đơn hàng / giây
30 client đồng thời

- Giám sát: pg_stat_statements, pg_stat_user_indexes, Grafana
- Dữ liệu: 3.033 SP, 35.000 đơn

Kết luận

- PostgreSQL = CSDL quan hệ + đa mô hình: một hệ thống thay cho search engine, cache, hàng đợi, vector DB
- 6 tính năng × 6 nghiệp vụ, đều có số đo: 2,5× – 150×, 50 client không bán quá kho
- Hướng phát triển: embedding CLIP theo ảnh, từ điển tiếng Việt, pg_cron, replication

Cảm ơn cô và các bạn đã lắng nghe!